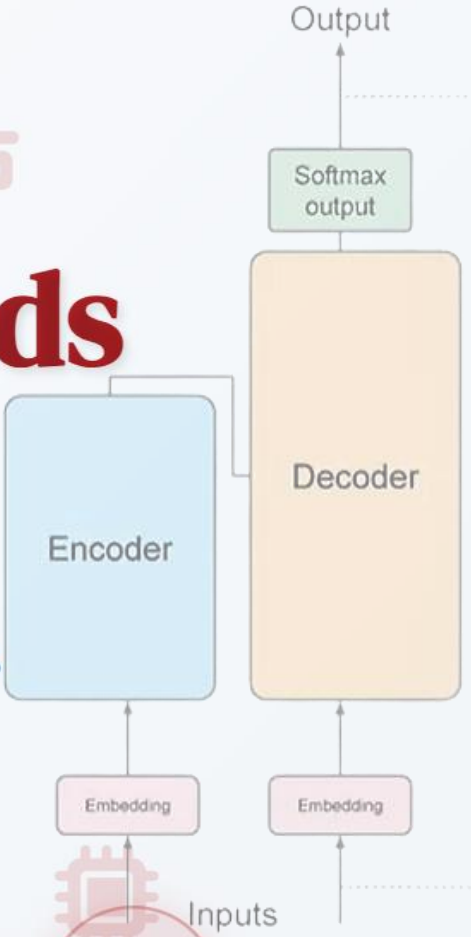


A HIGH LEVEL TOUR INTO

Ravana's 10 Heads and the Minds of Machines

Understanding the Architecture That Powers Transformers







Disclaimer

The content is curated from online/offline resources and used for educational purpose only.

About the Course

“

This course offers a myth-infused high-level overview into the mechanics of attention in modern machine learning. Drawing inspiration from the legendary figure of Ravana—we explore how machines mimic this **parallel processing** through **attention mechanisms**. You'll start with the basics of **word embeddings** and **positional encodings**, then journey through the core intuition of attention: **Queries, Keys, Values**, and the **weighted focus** that powers b. Through vivid analogies, intuitive visuals, and conceptual breakdowns, the course demystifies concepts like **self-attention**, **multi-head architectures**, and **context-aware** token representation.

Whether you're decoding the mind of Ravana or interpreting the focus of a neural network, this course equips you to understand—and build—intelligent systems that see the world from many angles at once.

”

Learning Objectives

You will learn in this lesson:

1) Understand the core concept of `attention` in `neural networks` through the lens of mythology and multi-perspective cognition.
2) Understand the idea of reducing a `n-dimensional vector` into known space like `3D` and represent the same in a more practical manner.
3) Break down the attention mechanism into `Query`, `Key`, and `Value` components and understand their mathematical interactions.
4) Visualize and interpret how `tokens` focus on one another using attention weights and heatmaps.
5) Explore the power of `multi-head attention` and how each head captures unique contextual relationships.
6) Differentiate between self-attention and cross-attention within encoder-decoder architectures.



Attention Is All You Need

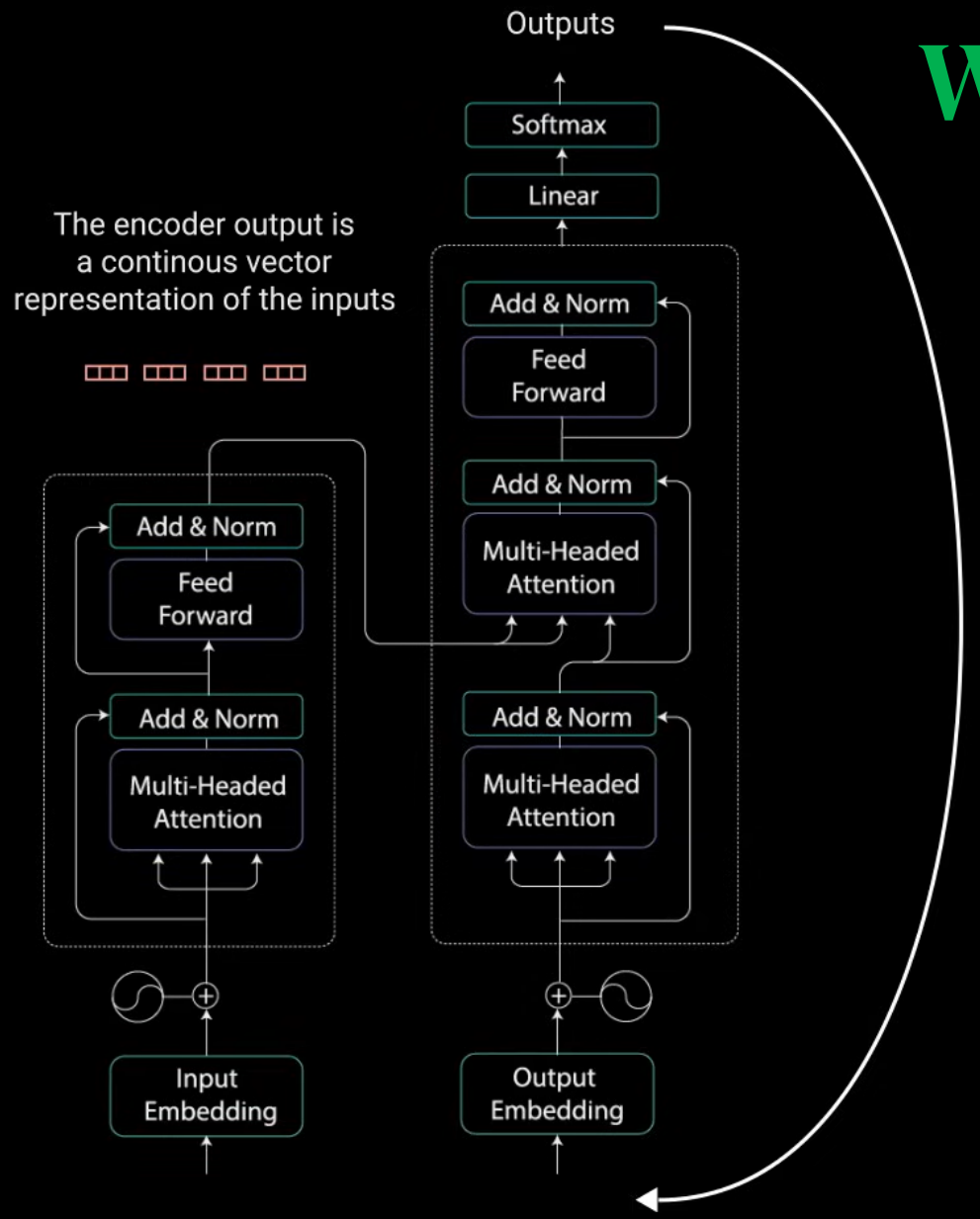
Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* † illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and



Where our story begins...

Feed previous outputs into the decoder recurrently until an "end of sentence" token, <end> is generated

Ravana's 10 Heads

Wisdom
Deep knowledge

Vision
Clear sight

Strategy
Cunning plans

Foresight
Future vision

Attention Heads

Syntax
Grammar rules

Semantics
Word meaning

Structure
Text organization

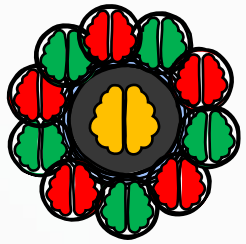
Context
Situational meaning

Introduction – The Legend & the Machine

Why Ravana?
Why Attention?

Multi-perspective processing
Each head: different cognitive faculty

Multi-head attention mechanism
Each head: learns different patterns



The Connection

Like Ravana's 10 heads processing information from multiple perspectives, attention mechanisms allow machines to **understand context** & **capture meaning** in language.

The Foundation – Word Embeddings

edunet foundation

From Language to Numbers: How Words Gain Meaning in Vectors

1. Machines cannot understand `raw words` — they need a `numeric representation`.

2. `Word embeddings` are dense vector representations of words trained to capture `semantic relationships`:

Semantic Closeness

```
"boy"
  →
- "man" + "woman"
  →
"girl"
```

"Boy" - "Man" + "Woman" ≈ "Girl"

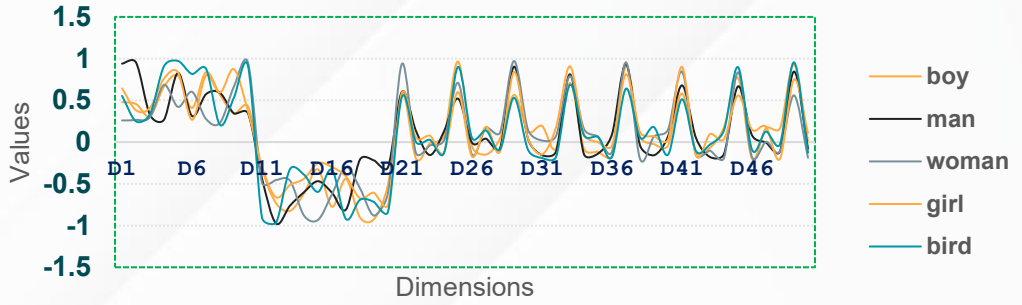
Similar meaning → Similar vectors

3. These vectors are the first inputs to the `attention mechanism`.

Embedding Types (Pre-trained)

- **Word2Vec** — Predictive models
- **GloVe** — Global vectors
- **BERT** — Contextual embeddings

Vector Space Visualization



Ravana's Analogy

Each word is a whisper into Ravana's mind, but the heads need to hear in **VECTORS**, not language.

The Missing Order – Positional Encoding

edunet foundation

Remembering Order: Injecting Meaning into Sequence

The Problem

Attention lacks order sensitivity

- ✕ Treats all words equally
- ✕ Ignores word position
- ✕ Sequence information lost

"Trainers train Saomya daily"
= "Saomya trains trainers daily"
Without position, meaning changes!

The Solution

Add Positional Encoding

- ✓ Preserves word order
- ✓ Encodes position information
- ✓ Enables sequence understanding

Consider the following sentence:
"Hi all! Welcome to the Transformers class by Saomya"

Tokens (9):

['Hi', 'all!', 'Welcome', 'to', 'the', 'Transformers', 'class', 'by', 'Saomya']

12194	722	26501	316	290	156303	744	656	132

Word Embeddings + **Positional Encoding**

Now words have both meaning and position!

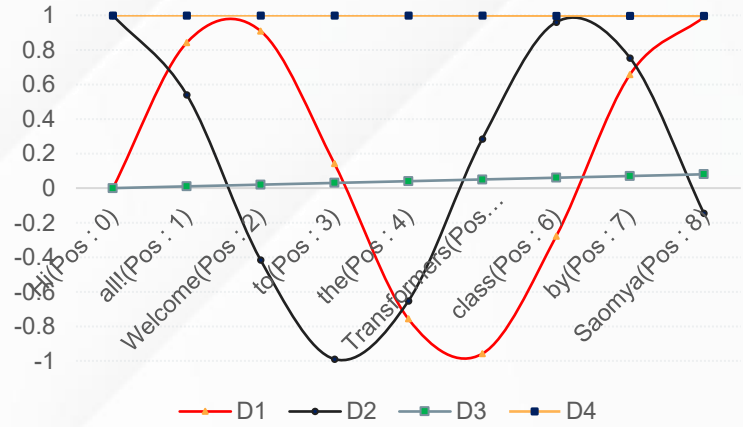
Fun Math Behind the Scene

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

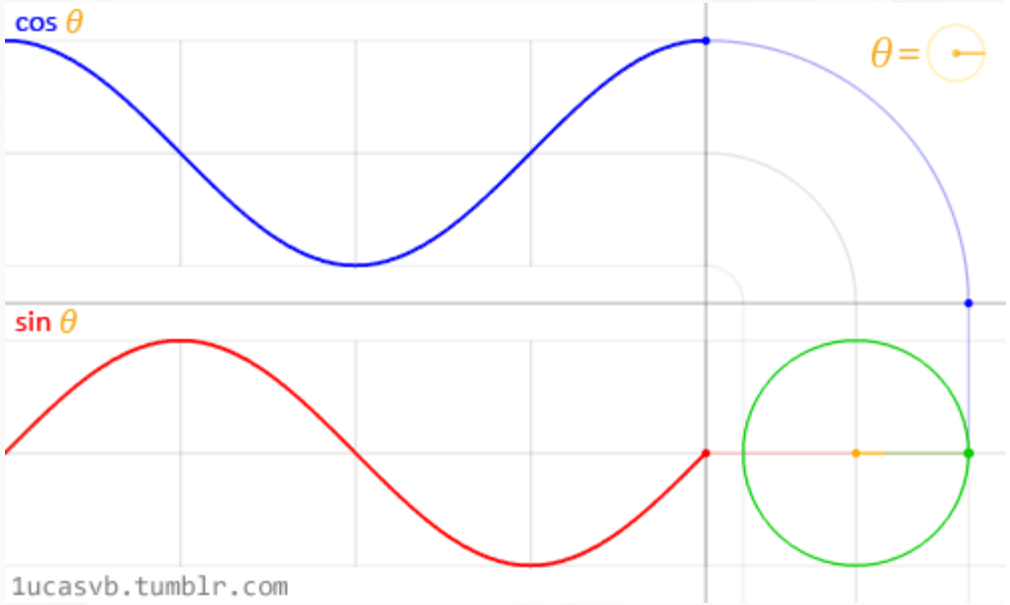
`pos` = position in sequence
`d` = embedding dimension
`i` = dimension pair index

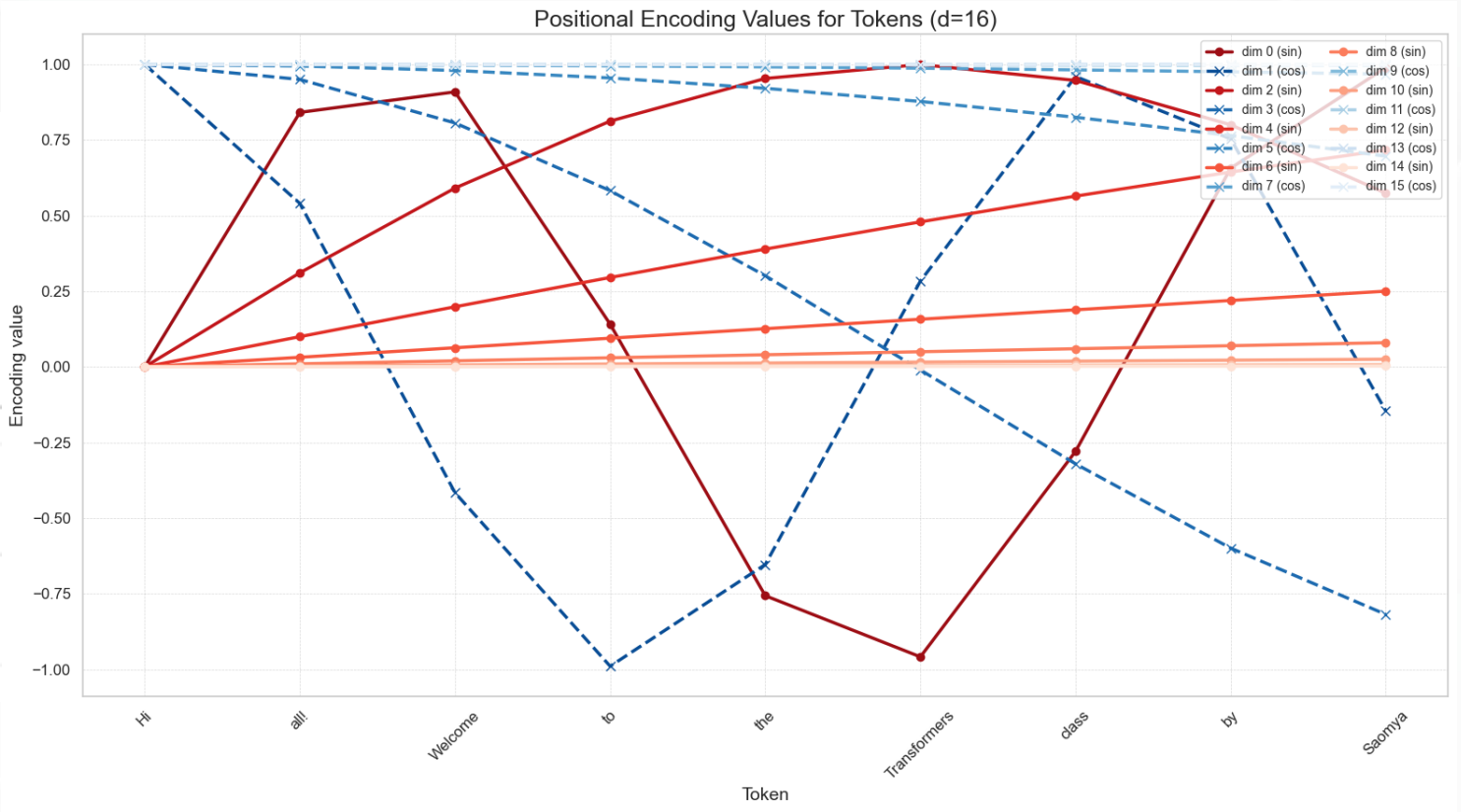
Positional Encoding Visualization



Ravana's Memory

Ravana doesn't just **hear** the words, he **remembers when** they were said.


cos θ
θ =
sin θ
1ucasvb.tumblr.com


Positional Encoding Values for Tokens (d=16)
Encoding value
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
Hi
all
Welcome
to
the
Transformers
class
by
Saomya
Token
dim 0 (sin)
dim 1 (cos)
dim 2 (sin)
dim 3 (cos)
dim 4 (sin)
dim 5 (cos)
dim 6 (sin)
dim 7 (cos)
dim 8 (sin)
dim 9 (cos)
dim 10 (sin)
dim 11 (cos)
dim 12 (sin)
dim 13 (cos)
dim 14 (sin)
dim 15 (cos)

The Final INPUT Form...

Words with Memory of Their Place

Input Formation Process

Token
"cat"

↓

Word Embedding
[0.2, -0.5, 0.7, ...]

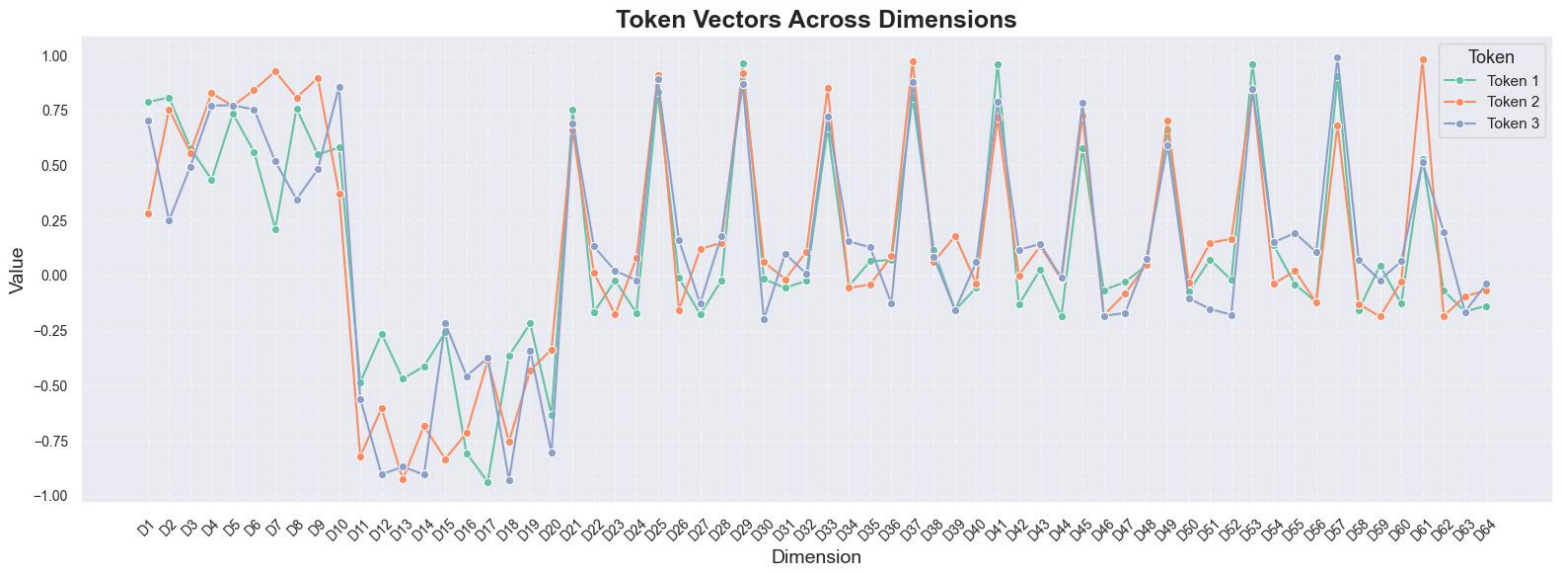
+

Positional Encoding
[0.1, 0.3, -0.2, ...]

=

Final Input Vector
[0.3, -0.2, 0.5, ...]

A hypothetical 64-Dimensional Vector Scenario Visualization



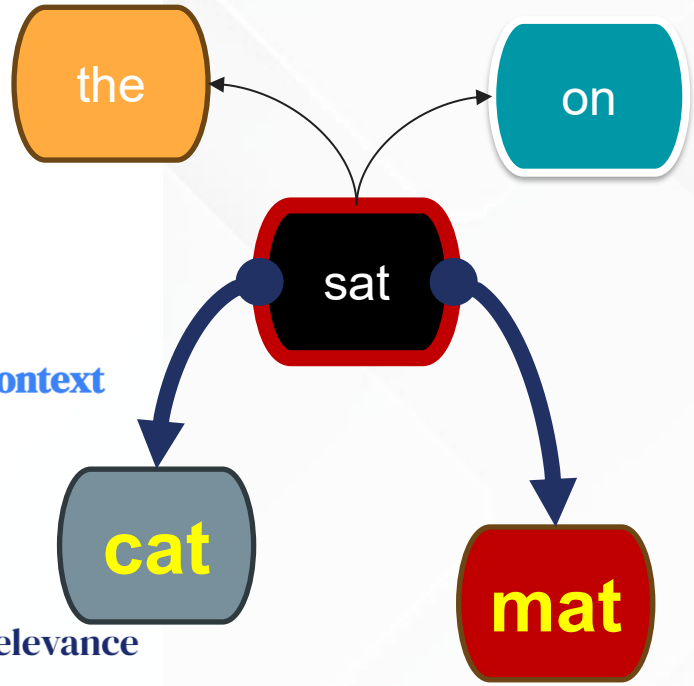
What is Attention? And Why should Machines Care?

Human Understanding

Non-linear & Contextual

We don't process words in isolation

Meaning depends on **surrounding context**



Attention Mechanism

Words **weigh** each other based on relevance

Focus on **important connections**

Multiple Perspectives

Who is speaking?
Focuses on speaker identity

What is meant?
Captures semantic meaning

What is implied?
Detects hidden meanings

Structure
Analyzes grammatical relations

Each head learns different patterns

Collective intelligence > single perspective

*Attention allows machines to process language the way humans do: **contextually** and **selectively***

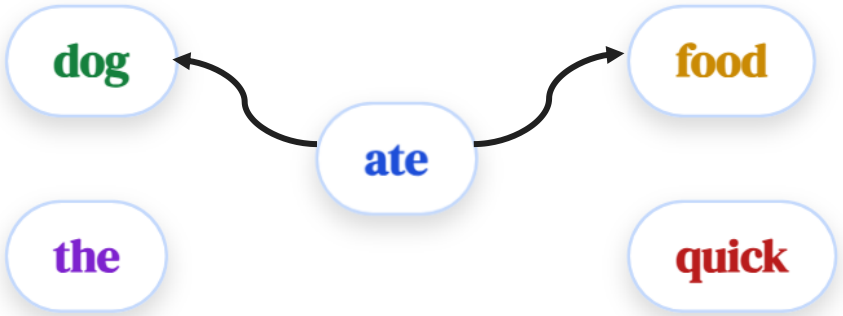
What Is Attention Doing, Really?

edunet foundation

Core Idea

Weighted Aggregation of Context

For each token, compute a **context-aware representation**



Context Vector = Σ weight × token vector

Attention Mechanism - Internally what happens

Listens to All
Each head processes all tokens in parallel

Filters Information
Assigns relevance scores to each token

Weights Importance
Creates weighted representation of context

Integrates Knowledge
Builds contextual understanding



*Attention creates a **context-aware representation** for each word, like Ravana's heads weighing the advice of others*

Query, Key, Value: The Language of Attention

edunet foundation

Attention Inputs

Query (Q)

What am I looking for?

"What information is relevant to me?"

Key (K)

What do I have to offer?

"What information might be relevant to others?"

Value (V)

What information do I carry?

"What actual content should be aggregated?"

Input Vector

[0.21, -0.34, ..., 0.15]

Query

[0.15, -0.22, ..., 0.08]

$W_q \times \text{Input}$

Key

[0.31, -0.18, ..., 0.42]

$W_k \times \text{Input}$

Value

[0.09, -0.27, ..., 0.19]

$W_v \times \text{Input}$

1. Each word vector is transformed by **learnable matrices.**
2. These matrices — W_q, W_k, W_v — project the input into query, key, and value spaces, enabling the attention mechanism to compare and weigh tokens.
3. These matrices are trained during **model optimization**, allowing the network to learn how best to attend to relevant information.
4. Each head transforms the same input into different Q, K, V vectors, allowing diverse perspectives on the same information. **What's that???**

Similarity Scoring – The Q · K^T Operation

Who Should I Pay Attention To?

Similarity Scoring

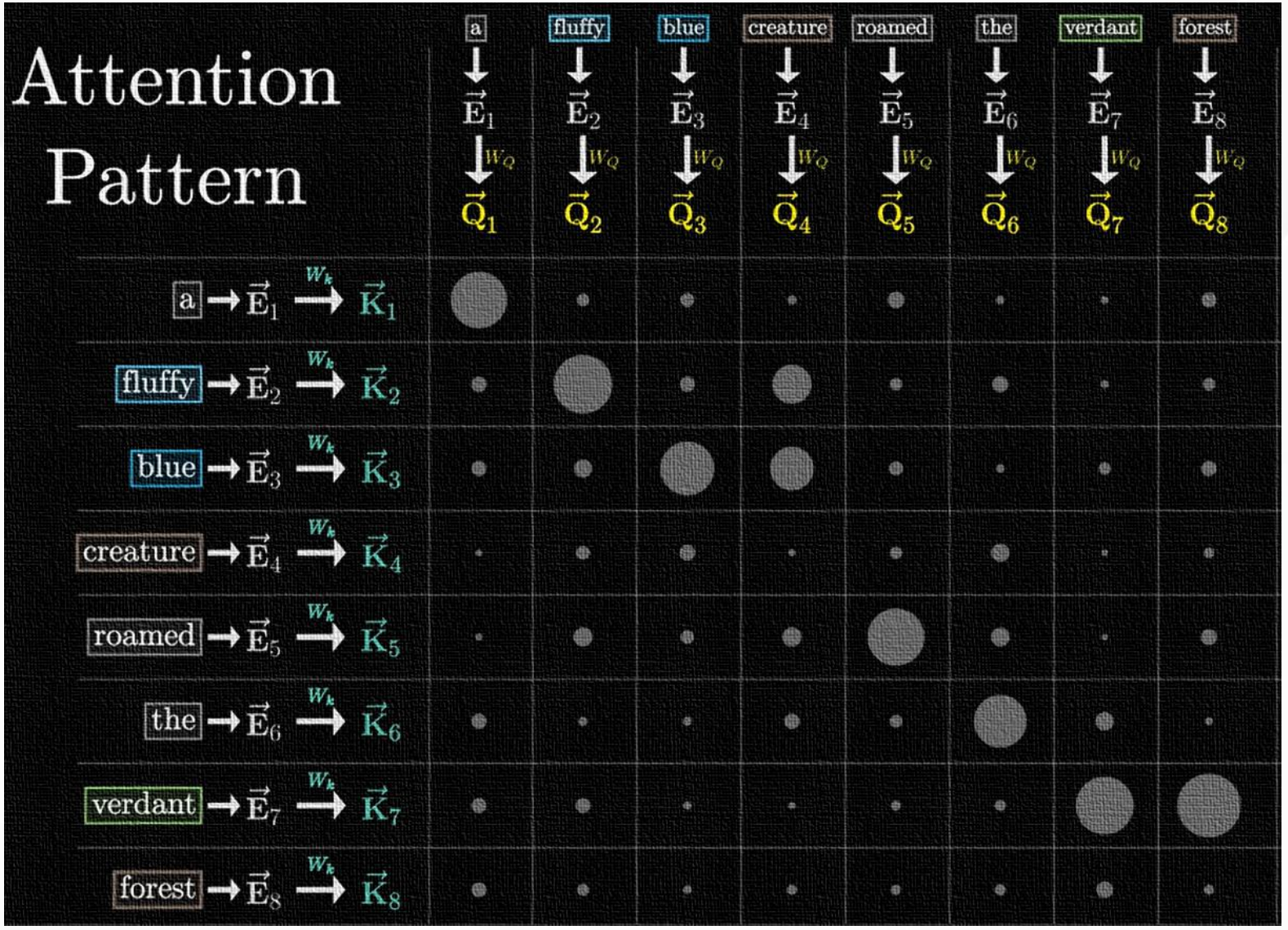
$$\text{Attention Score} = Q \cdot K^T$$

Query (Q) of current token
"What am I searching for?"

Key (K) of all tokens
"What do others offer?"

Dot Product (Q · K^T) Similarity Score
"How similar is the current token to others?"

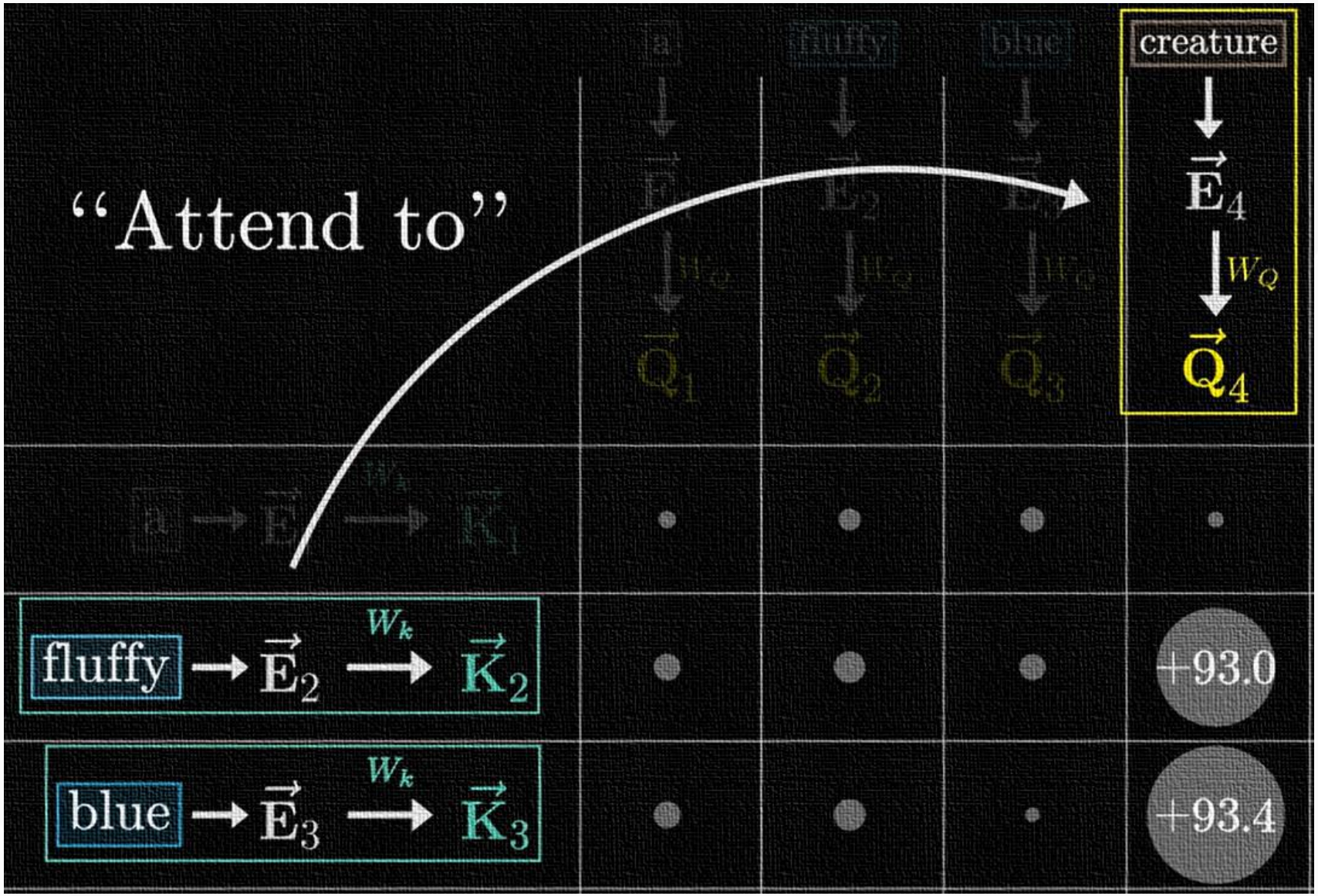
Softmax
"Convert scores into probabilities (attention weights)"



This is really what it means to ATTEND TO...

Something...

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$



Mathematical Foundation of Attention

From Words to Weights: The Math of Attention

Attention Inputs

Query (Q)
What am I looking for?

Key (K)
What do I have to offer?

Value (V)
What information matters?

Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

QK^T = Similarity score;
V = Value Matrix
$softmax$ = normalized focus;
d_k = dim of keys
Scaling factor prevents large dot products

Step-by-Step Breakdown

1. **Compute $QK^\top$**
   Dot product between queries and keys
   Higher score = more relevant

2. **Apply Softmax**
   Normalize scores into probabilities
   All weights sum to 1

3. **Multiply by Values**
   Weighted sum of value vectors
   Creates context-aware representation

4. **Output Vector**
   New representation of current token
   Informed by relevant context

Mathematical Mind

Each head performs complex calculations to determine which information is most relevant to the current query.

Softmax + Weighted Sum over Values

edunet foundation

Focus & Aggregate: Creating the Output Vector

Attention Process Flow

1 Apply Softmax

Normalize similarity scores

$$\text{softmax}(QK^T/\sqrt{d_k}) = [w_1, w_2, \ldots, w_n]$$

All weights sum to 1

2 Multiply by Values

Apply weights to value vectors

$$w_1V_1 + w_2V_2 + \ldots + w_nV_n$$

Each token contributes proportionally

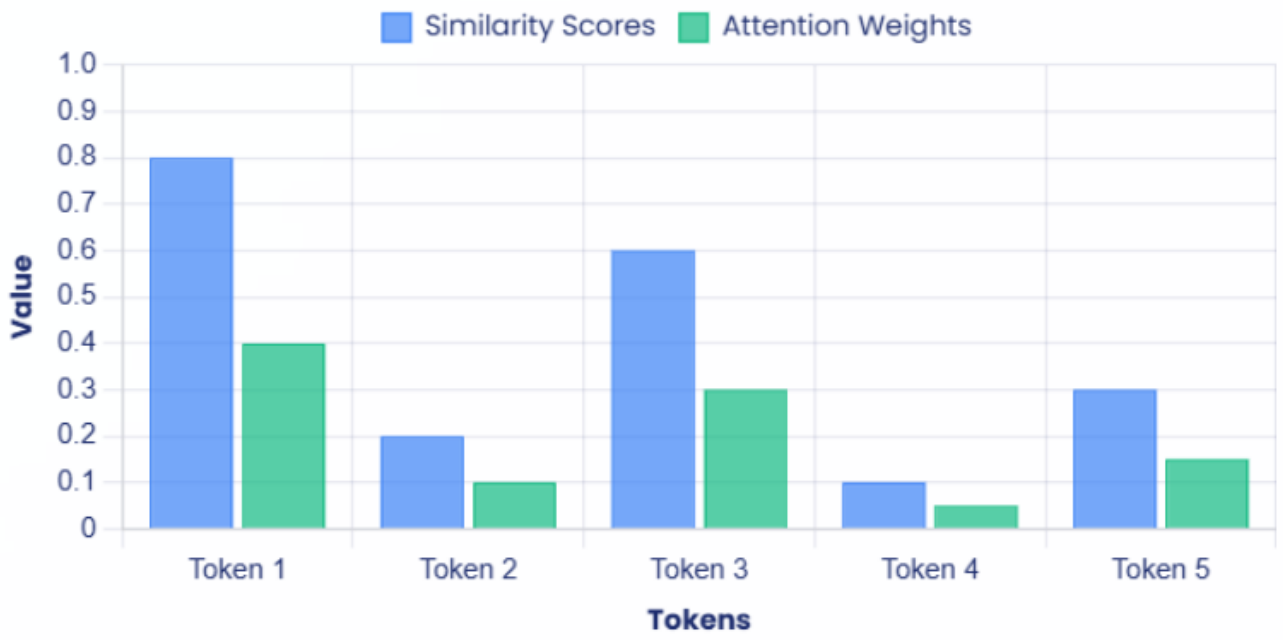
3 Output Vector

Context-aware representation

$$\text{Output} = \Sigma(w_i \times V_i)$$

New token representation in context

Weighted Aggregation Visualization



Scores	Weights	Output
[0.8, 0.2, 0.6, 0.1, 0.3]	[0.4, 0.1, 0.3, 0.05, 0.15]	[0.2, -0.3, 0.5, 0.1, 0.4]

Enter Multi – Head Attention Mechanism

edunet foundation

Ravana's 10 Heads: The Power of Many Views

Why Multiple Heads?

One Head ≠ Enough

Different heads focus on different features

Syntax
Grammar & structure

Semantics
Meaning & context

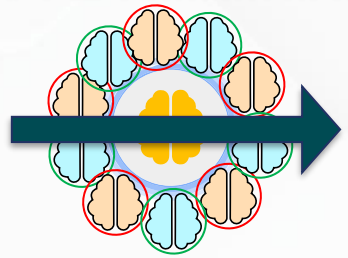
Relations
Entity connections

Pragmatics
Usage & intent

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_n)W^O$$

Input
Token Vector
(RAW)



Output
Context Vector
(REFINED)

Head Specializations

- Position
- Semantic
- Entity
- Syntax
- Pronoun
- Negation
- Coreference
- Modality

Benefits

Diverse Focus
Different perspectives

Richer Features
More expressive power

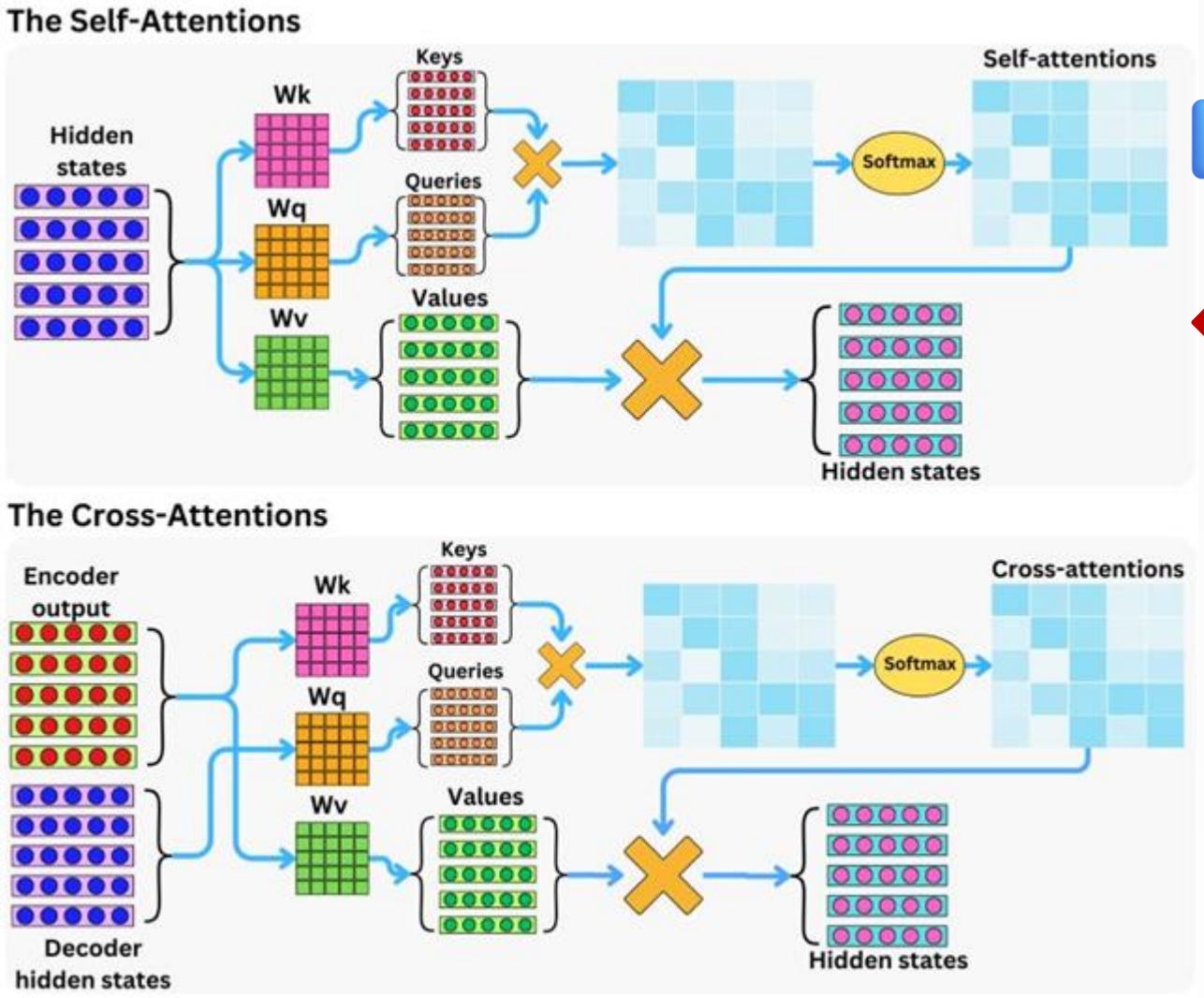
Better Generalization
Handles diverse inputs

Parallel Processing
Faster computation

Self – Attention VS Cross - Attention

edunet foundation

Thinking to Oneself vs. Listening to Others



Intra-sequence
Within the same input

Bidirectional
Sees past and future

A token attends to itself and others in the same sentence

Translation
Encoder-decoder models

Conditional
Guided generation

Decoder attends to encoder outputs (e.g., in translation)

Transformers – The Rise of Ravana

edunet foundation

How Attention Became the Foundation of Modern NLP

The Transformer Revolution

Before

RNNs & LSTMs

Sequential processing
Long-term dependencies

After

Transformers

Parallel processing
Global context

2017 — 2018 — 2019 — 2020+

Unprecedented Impact

- Faster training
- Scalable models
- Better understanding
- Zero-shot learning

Key Transformer Models

BERT

Bidirectional Encoder

Pre-training + fine-tuning

Google, 2018

GPT

Generative Pre-training

Autoregressive decoder

OpenAI, 2018-2020

T5

Text-to-Text Transfer

Unified framework

Google, 2019

BART

Bidirectional & Auto-Regressive

Denoising sequence-to-sequence

Facebook, 2019

Model	Release Year	Developed By	Notable Variant(s)	Parameters (Approx.)
BERT	2018	Google AI	BERT-Base / BERT-Large	110M / 340M
GPT	2018	OpenAI	GPT-1	117M
GPT-2	2019	OpenAI	GPT-2 Medium / Large / XL	345M / 762M / 1.5B
GPT-3	2020	OpenAI	GPT-3	175B
GPT-4	2023 (est.)	OpenAI	GPT-4	Not officially disclosed (~>500B est.)
T5	2020	Google Research	T5-Base / T5-Large / XXL	220M / 770M / 11B
BART	2020	Facebook AI (Meta)	BART-Base / BART-Large	139M / 406M

175B+
Parameters

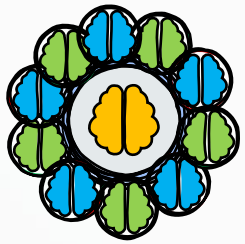
1000x
Training Speed

Few Applications

The NEED for Transformers



An intuitive [Transformer Playground]

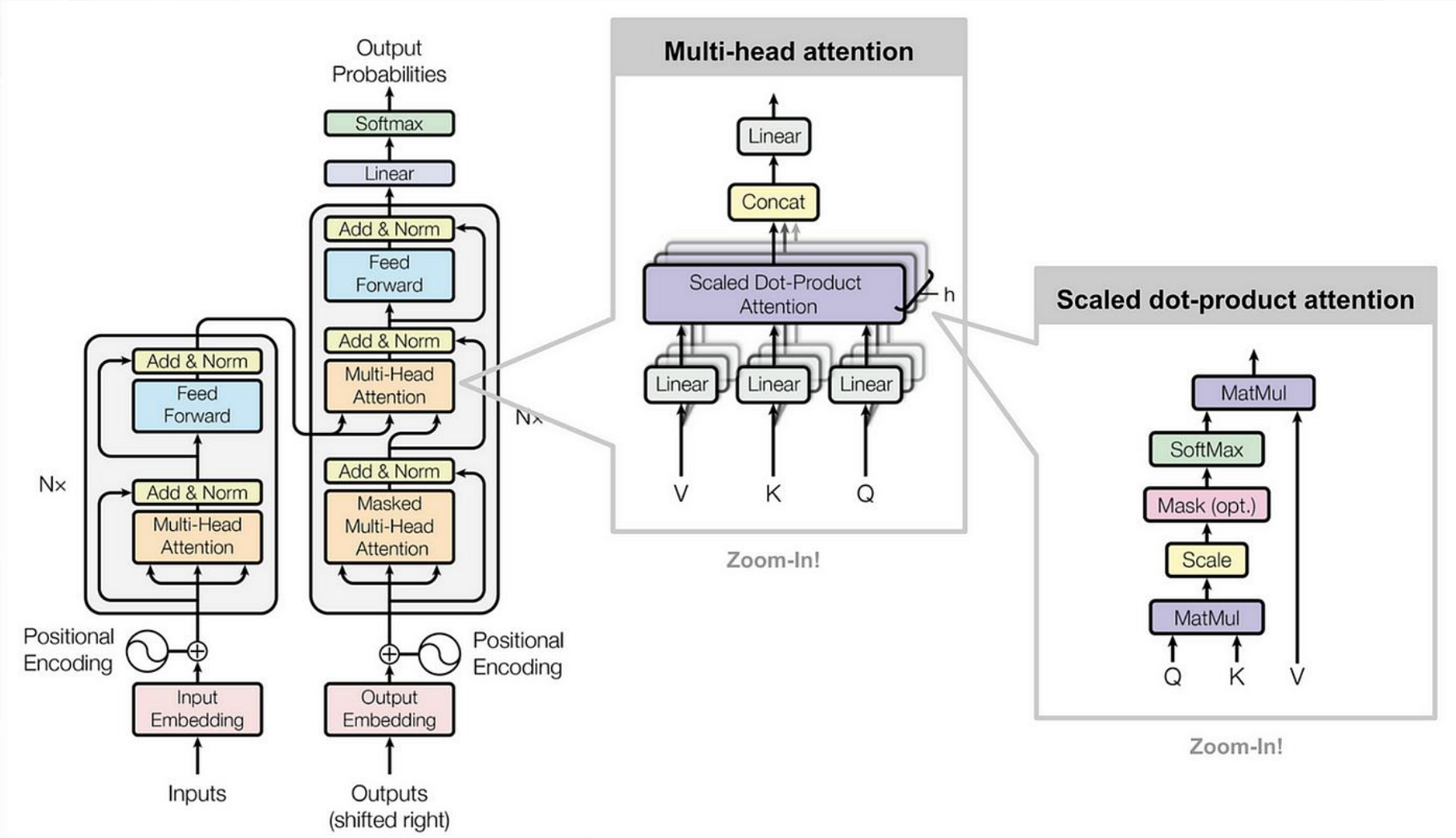


Ravana's Power

Just like Ravana's ten heads made him formidable, multi-head attention gives machines unparalleled power to understand and generate human language.

Final Thoughts

Peeking into the Giant Brain!



Thank You

CONTENT
Curated with

QUESTIONS



PLEASE

By Saomya Chaudhury